

Write logic for this:

1. A retail chain wants to forecast product sales using factors like holidays, promotions, location, and inventory. The dataset has 80 features. How should they select the most impactful ones?

Steps:

1. Preprocess data for cleaning and handling missing values.
 2. Perform Correlation analysis and remove the mostly correlated feature and least correlated features to Sales Target
 3. Use Recursive Feature Elimination (RFE) with any Regression models, to identify most important features.
 4. Alternatively you can use Lasso Regression to reduce less important features.
 5. Find R2 values to validate the best suited model and best features.
-

2. A podcast platform wants to recommend new podcasts to users based on the categories and hosts they follow. How should they design this content-based system?

Steps:

1. Encode podcast features like categories and hosts using OneHotEncoding or TF-IDF.
 2. Track user interactions with podcasts they follow or listen to, and assign weights to these actions.
 3. Build a user profile vector by combining weighted features from the podcasts the user likes.
 4. Represent each podcast as a feature vector with encoded categories and hosts.
 5. Calculate similarity scores between the user profile and podcast vectors using cosine similarity.
 6. Recommend the top podcasts with the highest similarity scores that the user has not heard yet.
-

3. An industrial firm wants to predict machine failure using sensor readings and system logs (200+ features). How do they choose which inputs to use?

Steps:

1. Start by cleaning the data and fixing any missing or wrong values.
2. Look at how each feature relates to machine failure and keep only those that show a strong connection.

3. Use a method like Recursive Feature Elimination (RFE) with a machine learning model to pick the important features step by step.
 4. Try a technique called Lasso regression that automatically finds and keeps only the needed features by shrinking others to zero.
 5. Test the chosen features using cross-validation to make sure the model predicts well on new data.
-

4. A fashion app wants to suggest outfits to users based on their previous likes, color preferences, and seasonal trends. How should they implement the recommendation?

Steps:

1. Encode outfit features like style, color, brand, and season using methods like TF-IDF or OneHotEncoding.
 2. Track user interactions such as likes and views, assigning weights to show what matters more.
 3. Create a profile vector for each user by combining weighted features from their interactions.
 4. Represent each new outfit as a feature vector.
 5. Compute similarity between the user profile and outfit vectors using cosine similarity or Approximate Nearest Neighbor (ANN) search.
 6. Rank and recommend the top N outfits with the highest similarity scores to the user.
-

5. A research lab wants to predict heart disease using patient data. The dataset has lab results, habits, and vitals. How should they select only the most relevant features?

Steps:

1. Clean and preprocess the data, handling missing values and encoding categorical variables.
 2. Use correlation analysis to find features strongly related to heart disease and remove weakly related ones.
 3. Apply Recursive Feature Elimination (RFE) with a suitable machine learning model to rank and select important features.
 4. Use Lasso regression to further refine the feature set by penalizing less important variables.
 5. Validate the selected features with cross-validation to ensure they help the model predict accurately.
-

6. A job portal wants to recommend job listings based on a user's resume and previous applications. How can they build a recommendation system?

Steps:

1. Extract key features from the user's resume such as skills, experience, education, and job titles using text processing techniques.
 2. Collect data on jobs the user has previously applied for or shown interest in.
 3. Create a user profile combining resume features and preferences from past applications.
 4. Represent job listings as feature vectors with similar attributes like required skills, job role, and location.
 5. Use similarity measures like cosine similarity or advanced methods like word embeddings to find jobs closely matching the user's profile.
 6. Rank jobs by their similarity scores and recommend the top options the user hasn't applied to yet, continuously updating the profile as they interact with new listings.
-

7. A smart home system wants to predict energy usage using readings from multiple sensors. With 100+ features, how do they reduce complexity?

Steps:

1. Clean and preprocess the sensor data, dealing with missing or noisy readings.
 2. Use statistical methods like correlation analysis to remove features that have little connection to energy usage.
 3. Apply dimensionality reduction techniques such as Principal Component Analysis (PCA) to capture the most important patterns in fewer components.
 4. Use feature selection methods like Recursive Feature Elimination (RFE) with a regression model to pick key features.
 5. Consider regularization techniques like Lasso regression to automatically shrink less important feature weights to zero.
 6. Validate the chosen features or components using cross-validation to ensure the model's performance remains strong while simpler.
-

8. An e-learning platform wants to suggest tutorials based on a learner's skill gaps identified from quizzes. How should they build this system?

Steps:

1. Analyze quiz results to identify which skills the learner lacks or needs improvement on.
2. Tag tutorials with the specific skills they teach using metadata or content analysis.
3. Create a learner profile highlighting the skill gaps based on quiz performance.

4. Match tutorials to the learner's gaps by comparing tutorial skill tags with the learner's missing skills.
 5. Rank tutorials that target the biggest gaps and recommend the top ones to the learner.
-

9. A social media app wants to predict post engagement (likes/comments) using post features, timing, and user activity. How do they select the right features?

Steps:

1. Start by cleaning the data and handling any missing or inconsistent values.
 2. Analyze the correlation between each feature (post content, timing, user activity) and engagement metrics like likes and comments.
 3. Use model-based methods such as Recursive Feature Elimination (RFE) with a regression model to find the most important features.
 4. Apply regularization techniques like Lasso regression to shrink less important feature weights to zero.
 5. Validate the selected features using cross-validation to ensure they improve prediction performance.
-

10. Now you want to create an app named attendance in the project and register it. write logic for this Django project

Steps:

1. Open your terminal and run the command: `python manage.py startapp attendance` to create the app.
 2. Open your project's settings.py file.
 3. Add 'attendance' to the INSTALLED_APPS list in settings.py to register the app.
 4. Create models, views, and templates inside the attendance app folder as needed.
 5. Run `python manage.py migrate` to apply any migrations related to the new app.
-